ASSIGNMENT FEEDBACK

Knowledge Acquisition System — Review

For: Amrith · Repo: 0Amrith0/rretap · Date: 5 Sep 2026 · Includes review of your Architecture and Design PDFs

64 / 100 — The system runs and most claims are verifiable. The score is limited by OKF validity failures, a missing re-run proof, and documentation that does not match the code.

Thank you for the submission. I tested the system before I scored it: I ran the scripts again, sent test payloads to the hook, and compared facts against the cited pages. The two largest gaps are small changes with high point value. Treat this document as a work list for a second review.

VERIFIED CORRECT BEHAVIOR

- The re-run check works: a new fetch on 2026-09-05 matched the stored hash and skipped all later stages.
- I compared more than 30 facts against their live sources. All matched. I did not find a false fact.
- The hook is registered correctly and passed 10 live tests: it blocks bad writes and passes unrelated commands.
- The limitations in your design doc match the code, and all 56 tests pass with behavior checks, not constant checks.

Score vs. criteria

CATEGORY	WEIGHT	SCORE	WHY
Problem Decomposition	20%	**16 / 20**	10+ subsystems with clear boundaries and tested edge cases; three source types merged into shared documents. I removed points for two unsolved sub-problems: Discover only re-checks a fixed list, and a fixed 12-key taxonomy does the dedup instead of concept matching. You documented both limits.
Claude Code Usage	20%	**12 / 20**	Clear subagent prompts, and the hook passed all 10 live tests. But the three skills do not load: Claude Code needs `SKILL.md` files in skill folders; your skills are flat files. I confirmed this in a live session. CLAUDE.md describes features the code does not have. One commit holds 27 files, so history does not show step-by-step work.
Agent Architecture	20%	**12 / 20**	Subagent contracts are clear: JSON-only output, read-only tools, correct code/Claude split. I removed points because no command or skill sequences the five stages, so the brief example command was never shown to work. hash.js also writes state before later stages run; a crash mid-run hides the update.
Code Quality	15%	**12 / 15**	Scripts are consistent: data on stdout, logs on stderr, no external dependencies. I removed points for three defects: npm test fails on Node 20 and older; fetch has no timeout or retry; write-okf.js logs an update when the content did not change.
Knowledge Quality	15%	**6 / 15**	More than 30 facts matched their live sources, and all 63 facts carry a citation. But 12 of 12 files fail the OKF v0.1 hard rule: frontmatter must contain a non-empty `type` field. index.md and log.md have no frontmatter; your own validator rejects them, and write-okf.js ships them anyway. Related links exist only in frontmatter. Two of your 12 topics have no file.
Documentation	10%	**6 / 10**	The design doc limitations match the code. I removed points because two required Design Doc sections are missing, the Example Ingestion Run deliverable is absent, and CLAUDE.md and README contain claims the code does not support.
Total	**100%**	**64 / 100**	**Below the level I expect for this assignment. The two largest gaps are small changes with high point value. Fix them first.**

How to improve it (priority order)

P0 — do first, largest effect · P1 — large effect · P2 — necessary · P3 — polish

P0 Make the bundle valid OKF v0.1

I saw: The brief defines one hard rule: every `.md` file needs frontmatter with a non-empty `type` field. None of your 12 files has `type`. index.md and log.md have no frontmatter; your own validator rejects them, but `write-okf.js` ships them without validation.

Why it matters: The brief puts valid OKF output first, before safe re-runs. Your facts are accurate, but the bundle fails the one required check.

Do this: Add `type:` to the 10 topic files. Add frontmatter with `type` to index.md and log.md. Make `write-okf.js` validate every file it writes. Add the recommended fields: `description`, `tags`, `timestamp`. About two hours of work; most Knowledge Quality points depend on it.

P0 Deliver the Example Ingestion Run with the re-run proof

I saw: Deliverable 5 asks for a `RUN.md` : commands in order, output samples, and a git diff that shows a second run changed nothing. No `RUN.md` exists. log.md has one no-change line for one source, which does not support your design doc claim of two or three runs.
Why it matters: The core requirement is proof that a second run changes nothing. Your system probably passes; the proof is missing.
Do this: Run the full ingest two times. Save the commands and the empty `git diff knowledge/` in `RUN.md` . Commit both.

P1 Make the skills load

I saw: Your skills are files named `.claude/skills/<name>.md` . Claude Code loads skills from `.claude/skills/<name>/SKILL.md` . In a live session none of the three appeared; the Read fallback in your agent prompts did all the work.
Why it matters: Skills are one of the four building blocks in the brief. Your one-file-rules plan needs them to load.
Do this: Move each to `.claude/skills/<name>/SKILL.md` with frontmatter. Remove the fallback text. Remove the duplicated confidence rules from `validator.md` and `merger.md` .

P1 Make the documentation match the code

I saw: CLAUDE.md says fetch.js uses Playwright MCP; no Playwright code exists, and your PDF says "no MCP tooling". It says the hash stage writes the no-change log line; no script does. It points to a build-plan document the repo does not contain. It says the hook runs `validate-schema.js` ; settings.json runs a wrapper.
Why it matters: CLAUDE.md is the first input for the agent and a future maintainer. Wrong statements cause wrong behavior.
Do this: Check each statement in CLAUDE.md and README against the code. Remove each claim you cannot show with a command.

P1 Encode the orchestration

I saw: No command, script, or skill runs the five stages in order. The steps exist only as text in CLAUDE.md and script comments, so the brief example command was never shown to work.
Do this: Add `.claude/commands/ingest.md` : it runs the stages in order and handles a stage failure. Save its output in `RUN.md` .

P2 Fix the re-run integrity defects

I saw: `hash.js` writes the new hash before the later stages run; a crash mid-run hides the update. `write-okf.js` logs an "updated" line when the content did not change. fetch has no timeout or retry; a GitHub rate limit stops the run.
Do this: Write state only after publish finishes. Skip the log line when the content is unchanged. Add a timeout and one retry to fetch.

P2 Fix the test command

I saw: `npm test` needs Node 21+ (the glob passes through sh). The brief states Node 18+. package.json has no `engines` field.
Do this: Use `node --test tests/` . Add `"engines": {"node": ">=18"}` .

P2 Fill the coverage and reflection gaps

I saw: Two of your 12 topics (Sponsored-Display, Amazon-DSP) have no file. AMC, Marketing Stream, and MCP servers from the brief scope list are absent. Related links exist only in frontmatter. The Design Doc lacks "what surprised you" and "what you learned".
Do this: Ingest one more source to fill the two missing topics. Add links in body text between related files. Write the two missing sections.

P3 Polish

I saw: One 27-file commit; later messages do not explain the changes. The hook command uses a relative path. `write-okf.js` prints a stack trace; the other scripts print one clean line. ACOS-ROAS-Metrics.md does not define ROAS.
Do this: Make small commits with clear messages. Use `$CLAUDE_PROJECT_DIR` in the hook command. Make error handling consistent. Add a CI workflow.

These changes are small. Send the updated repository for a second review.